Министерство науки и вышего образования Российской Федерации

Федеральное государственное бюджетное образовательное учреждение высшего образования

« Московский педагогический государственный университет »

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

Выполнил студент группы ЗОМ34-ПВШ2616: **Золотухин Андрей Александрович**

Ссылка на репозиторий: https://github.com/CorgiPuppy/dslp-labs

Принял: Жданов Сергей Александрович

Дата сдачи: 23.10.26

Москва

2026

СОДЕРЖАНИЕ

1. Задание 1 3
   1.1. Ознакомление с примерами из лекции 3
2. Задание 2 9

1. Задание 1

1.1. Ознакомление с примерами из лекции

```cpp
// check_prime.cpp
// C++ программа для проверки простоты числа (рекурсивная)
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей
Александрович

#include <iostream>
#include <cmath>

// рекурсивная функция для проверки числа на простоту
int checkPrime(int, int);

int main() {
  int num;
  std::cout << "Enter your number: ";
  std::cin >> num;

  if (checkPrime(2, num) == 0)
    std::cout << "Number " << num << " is a primer." << std::endl;
  else
    std::cout << "Number " << num << " isn't a primer." << std::endl;

  return 0;
}

// принимаю 2 (минимальное простое число) и само рассматриваемое число
int checkPrime(int i, int num) {
  // если мы дошли до самого числа, то, очевидно, у него не было делителей,
кроме него самого и 1, поэтому выводим 0 (то есть правду)
  if (num == i) return 0;
  // если число делится на число от 2 до num -1, то оно имеет хотя бы один
третий делитель (помимо 1 и самого числа), значит выводим, что число не
простое
  else if (num % i == 0) return 1;
  // иначе идём рассматривать следующий делитель
  else return checkPrime(i + 1, num);
}
```

```
Enter your number: 50
Number 50 isn't a primer.
```

```
Enter your number: 53
Number 53 is a primer.
```

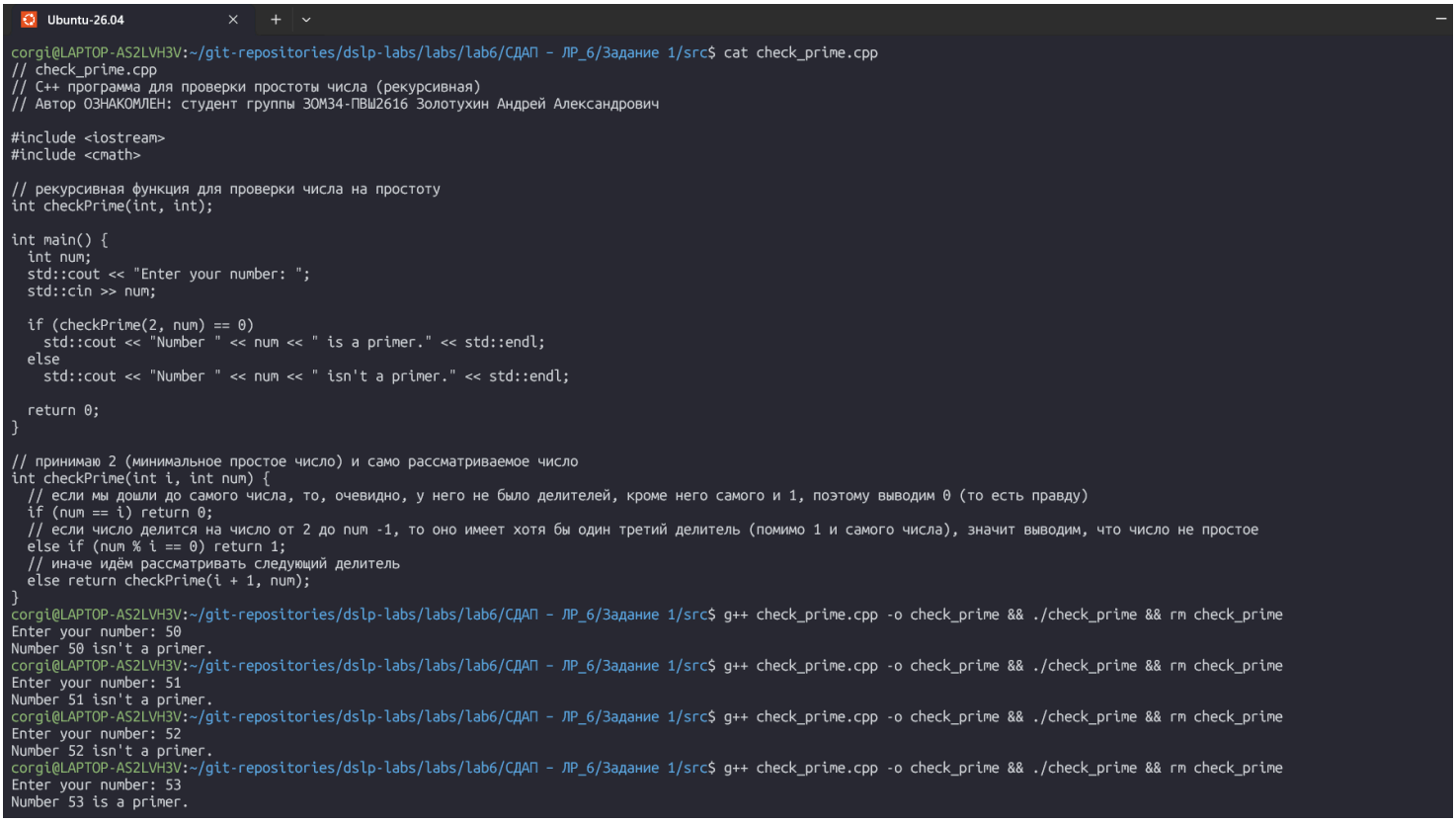


Рис. 1: Авторская иллюстрация программы check_prime.cpp

```cpp
// gen_primes.cpp
// C++ программа с модулем - функцией генерации массива простых чисел, меньших n
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей Александрович

#include <cmath>

/*  функция возвращает количество простых чисел
 *  в массиве primes из n элементов
 */
int gen_primes(int *primes,int n){
  int m, p, pk;
  int k = (int)std::sqrt(n + 1);
  int *buf = new int[n];

  // заполняем буфер всеми числами от 2 до n
  buf[0] = 2;
  for (int i = 1; i < n; i++)
    buf[i] = i + 1;
```

```
  p = 0;
  while(p < k){
    m = p + 1;
    while(m < n) {
      if (buf[m] % buf[p] == 0) buf[m] = -1;
      m++;
    }
    pk = 0;
    for (int i = 0; i < n; i++)
      if (buf[i] != -1)
        primes[pk++] = buf[i];
    for (int i = 0; i < n; i++)
        buf[i] = (i < pk) ? primes[i] : -1; // сократил до тернарного
оператора
    p++;
  }

  delete[] buf; // в примере не было, но я добавил для освобождения памяти

  return pk;
} // end function
```

```
// tab_primes1.cpp
// C++ программа для таблицы простых чисел, меньших n
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей
Александрович

#include <iostream>
#include <iomanip>
#include <cmath>
int main(){
  int m, p, pk, n;
  std::cout << "n = ";
  std::cin >> n;
  std::cout << "Numbers: " << n << std::endl;
  int k = (int)std::sqrt(n + 1);

  int *buf = new int[n];
  int *primes = new int[n];

  // заполнение буфера числами от 2 до n
  buf[0] = 2;
  for (int i = 1; i < n; i++){
    buf[i] = i + 1;
  }
  p = 0;
```

```
  while(p<k){
    m = p + 1;
    while(m < n) {
      if (buf[m] % buf[p] == 0) buf[m] = -1;
      m++;
    }
    pk = 0;
    for (int i = 0; i < n; i++) {
      if (buf[i] != -1) {
        primes[pk] = buf[i];
        pk++;
      }
    }

    for (int i=0;i<n;i++)
      buf[i] = (i < pk) ? primes[i] : -1;
    p++;
  }

  // Вывод всех простых чисел, меньших n
  std::cout << "Primes: " << pk << std::endl;
  for (int i = 0;i < pk; i++){
    std::cout << std::setw(4) << primes[i];
    if (i % 9 == 0 && i != 0) std::cout << std::endl; // перевод строки после
9 числа
  }
  std::cout << std::endl;

  return 0;
}
```

```
n = 70
Numbers: 70
Primes: 19
   2   3   5   7  11  13  17  19  23  29
  31  37  41  43  47  53  59  61  67
```

```
// tab_primes2.cpp
// C++ программа для таблицы простых чисел
// Автор ОЗНАКОМЛЕН: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей
Александрович

#include <iostream>
```

```cpp
#include <iomanip>
#include <cmath>

// Модуль-функция генерации массива простых чисел меньших n
// функция возвращает количество простых чисел в массиве primes из n элементов
int gen_primes(int *primes, int n) {
  int m, p, pk;
  int k = (int)std::sqrt(n + 1);
  int *buf = new int[n];

  // заполняем буфер всеми числами от 2 до n
  buf[0] = 2;
  for (int i = 1; i < n; i++)
    buf[i] = i + 1;
  p = 0;
  while(p < k){
    m = p + 1;
    while(m < n) {
      if (buf[m] % buf[p] == 0) buf[m] = -1;
      m++;
    }
    pk = 0;
    for (int i = 0; i < n; i++)
      if (buf[i] != -1)
        primes[pk++] = buf[i];
    for (int i = 0; i < n; i++)
        buf[i] = (i < pk) ? primes[i] : -1; // сократил до тернарного
оператора
    p++;
  }

  delete[] buf; // в примере не было, но я добавил для освобождения памяти

  return pk;
} // end function

int main() {
  int nprimes, n;
  std::cout << "n = "; // ввод желаемого n
  std::cin >> n;
  std::cout << "Numbers: " << n << std::endl;
  int* primes = new int [n];
  nprimes = gen_primes(primes, n);
  std::cout << "primes: "<< nprimes << std::endl;
  for (int i = 0; i < nprimes; i++){
    if(i % 9 == 0 && i != 0) std::cout << std::endl;
```

```
    std::cout << std::setw(4) << primes[i];
  }
  std::cout << std::endl;
  return 0;
}
```

```
n = 72
Numbers: 72
primes: 20
   2   3   5   7  11  13  17  19  23
  29  31  37  41  43  47  53  59  61
  67  71
```

2. Задание 2

```
// primes_utils.h
// Модуль для работы с простыми числами
// Автор: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей Александрович

#ifndef PRIMES_UTILS_H
#define PRIMES_UTILS_H

#include <vector>

// Проверка числа на простоту
bool isPrime(int num);

// генерация массива простых чисел, меньших n
// возвращает количество найденных простых чисел
int gen_primes(int* primes, int n);

// функция для нахождения всех простых делителей числа N с учётом кратности
std::vector<int> getPrimeFactors(int N);

// функция, чтобы посчитать сколько раз простое P входит в факторизацию числа
x
long long countPower(long long x, long long P);

// функция, чтобы посчитать сколько раз простое P встречается во всех числах
от L до R
long long countInRange(long long L, long long R, long long P);

#endif
```

```
// primes_utils.cpp
// Реализация модуля работы с простыми числами
// Автор: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей Александрович

// импортирую головной файл, где были инициализированы эти модули
#include "../include/primes_utils.h"

#include <cmath>
#include <vector>

// рекурсивная функция для проверки числа на простоту
int checkPrimeRec(int i, int num) {
  // если мы дошли до самого числа, то, очевидно, у него не было делителей,
кроме него самого и 1, поэтому выводим 0 (то есть правду)
  if (num == i) return 0;
```

```
  // если число делится на число от 2 до num -1, то оно имеет хотя бы один третий делитель (помимо 1 и самого числа), значит выводим, что число не простое
  if (num % i == 0) return 1;
  // иначе идём рассматривать следующий делитель
  return checkPrimeRec(i + 1, num);
}
bool isPrime(int num) {
    if (num <= 1) return false;
    return checkPrimeRec(2, num) == 0;
}

// Генерация простых чисел
int gen_primes(int* primes, int n) {
    if (n < 2) return 0;

    int k = (int)std::sqrt(n + 1);
    int* buf = new int[n];

    // заполняем буфер числами от 2 до n
    buf[0] = 2;
    for (int i = 1; i < n; i++)
        buf[i] = i + 1;

    int p = 0;
    int pk = 0;

    while (p < k) {
        // вычёркиваю кратные текущему простому
        int m = p + 1;
        while (m < n) {
            if (buf[m] != -1 && buf[m] % buf[p] == 0)
                buf[m] = -1;
            m++;
        }

        // собираю оставшиеся числа
        pk = 0;
        for (int i = 0; i < n; i++) {
            if (buf[i] != -1)
                primes[pk++] = buf[i];
        }

        // обновляю буфер
        for (int i = 0; i < n; i++)
            buf[i] = (i < pk) ? primes[i] : -1;
```

```
        p++;
    }

    delete[] buf; // освобождаю память

    return pk;
}

// Модуль-функция для нахождения простых делителей с кратностью
std::vector<int> getPrimeFactors(int N) {
    std::vector<int> factors;
    if (N <= 1) return factors;

    // двойка
    while (N % 2 == 0) {
        factors.push_back(2);
        N /= 2;
    }

    // нечётные делители
    for (int i = 3; i * 1LL * i <= N; i += 2) {
        while (N % i == 0) {
            factors.push_back(i);
            N /= i;
        }
    }

    // оставшееся простое
    if (N > 1)
        factors.push_back(N);

    return factors;
}

// модуль-функция для подсчёта степени простого в одном числе
long long countPower(long long x, long long P) {
    long long counter = 0;
    while (x > 0 && x % P == 0) {
        counter++;
        x /= P;
    }
    return counter;
}

// модуль-функция для подсчёта в диапазоне [L, R]
```

```
long long countInRange(long long L, long long R, long long P) {
    long long total = 0;
    for (long long i = L; i <= R; i++)
        total += countPower(i, P);
    return total;
}
```

```
/* Автор: студент группы ЗОМ34-ПВШ2616 Золотухин Андрей Александрович
 * СДАП Лабораторная работа 6
 * Язык: C++
 */

#include "../include/primes_utils.h"

#include <iostream>
#include <vector>

int main() {
  /*
   * Задача 1
   * нахождение простых делителей числа (с повторениями) или NOT
   */
  std::cout << "--- Задача 1 ---" << std::endl;
  std::cout << "Введите число N: ";
  int N;
  std::cin >> N;

  if (isPrime(N)) {
    std::cout << "Результат: NOT (число простое)" << std::endl;
  } else {
    std::vector<int> factors = getPrimeFactors(N);
    std::cout << "Простые делители: ";
    for (int i = 0; i < factors.size(); i++) {
      std::cout << factors[i];
      if (i + 1 < factors.size()) std::cout << " ";
    }
    std::cout << std::endl;
  }
  std::cout << std::endl;
  /*
   * Конец задачи 1
   */

  /*
   * Задача 2
   * сколько раз простое P встречается в разложении чисел [L, R]
```

```
   */
  std::cout << "--- Задача 2 ---" << std::endl;
  std::cout << "Введите L, R и P (P - простое): ";
  long long L, R, P;
  std::cin >> L >> R >> P;

  long long result = countInRange(L, R, P);
  std::cout << "Количество вхождений P в разложениях чисел от " << L << " до "
<< R << ": " << result << std::endl;
  /*
   * Конец задачи 2
   */
  return 0;
}
```

```
--- Задача 1 ---
Введите число N: 5
Результат: NOT (число простое)

--- Задача 2 ---
Введите L, R и P (P - простое): 2 8 2
Количество вхождений P в разложениях чисел от 2 до 8: 7
```

```
--- Задача 1 ---
Введите число N: 10
Простые делители: 2 5

--- Задача 2 ---
Введите L, R и P (P - простое): 25 50 7
Количество вхождений P в разложениях чисел от 25 до 50: 5
```